Cloud-Based Inventory Management System
Project Report

1. Introduction

This project will create and deploy a cloud-based inventory management system using AWS. The system will focus on building a secure, scalable and serverless backend to perform key inventory actions, including add items, decrease or increasing the item's stock count, and assigning files. This will rely on HTTPS APIs and AWS managed services, with no need for servers or manual scaling.

2. Objectives

- To build a serverless back-end system using AWS managed services.
- To build secure user authentication and authorization with Amazon Cognito.
- To build an HTTP based API layer on using AWS API Gateway (HTTP API) that is connected to AWS Lambda.
- To use Amazon DynamoDB to store inventory, stock, and transaction data.
- To use Amazon S3 to store images and file exports.
- To enable/implement monitoring and log data to enable future availability and security.

3. AWS Services Used

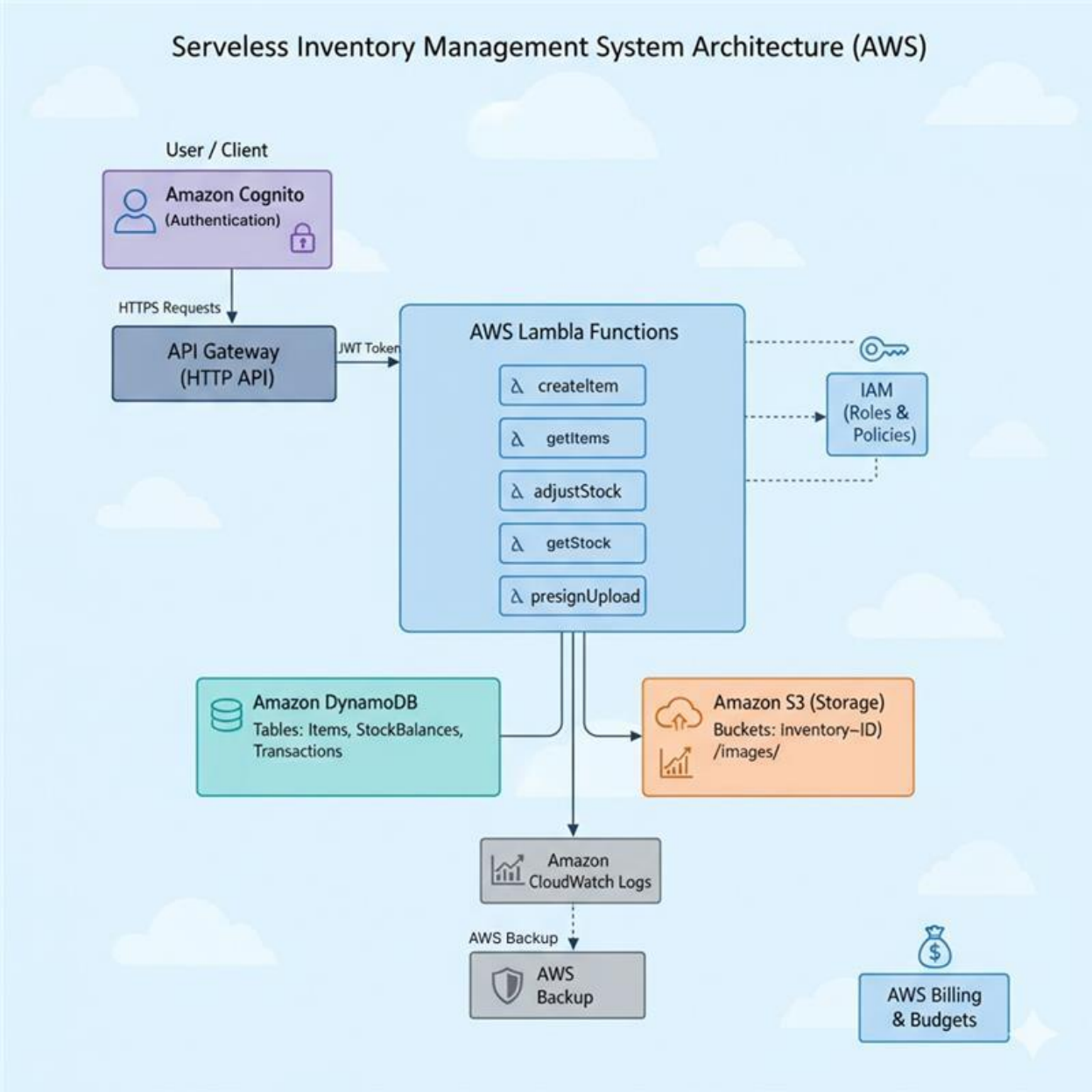
AWS Service	Purpose / Function
IAM (Identity and Access Management)	Controlled access by creating users, roles, and policies.
Cognito	Handled user registration, login, and token-based authentication (JWT).
API Gateway (HTTP API)	Exposed HTTPS endpoints to interact with backend Lambda functions.
AWS Lambda	Hosted the business logic for item and stock management.
DynamoDB	NoSQL database storing items, stock levels, and transaction records.
S3 (Simple Storage Service)	Stored images and exported inventory reports.
CloudWatch	Monitored logs, metrics, and Lambda execution.
Billing & Budgets	Set cost alerts to monitor AWS expenses.
AWS Backup & PITR (Point-in-Time Recovery)	Ensured data protection and recovery for DynamoDB.

4. System Architecture

The system uses a **fully serverless, HTTPS-based architecture** powered by AWS managed services. API Gateway (HTTP API) provides a lightweight and cost-efficient way to expose backend Lambda functions over secure HTTPS endpoints.

Workflow Overview

1. A user logs in via **Cognito Hosted UI** and receives a secure JWT token.
2. The user sends an HTTPS request to **API Gateway (HTTP API)**.
3. API Gateway verifies the token and triggers the corresponding **Lambda function**.
4. Lambda reads or writes data in **DynamoDB**, or stores/retrieves objects from **S3**.
5. All actions and errors are logged in **CloudWatch** for monitoring.



5. Implementation Steps

Step 1: IAM User and CLI Configuration

- Created IAM user dev-admin with **Administrator Access** (for setup phase).
- Enabled **MFA (Multi-Factor Authentication)**.
- Configured AWS CLI locally using aws configure.
- Verified credentials with aws sts get-caller-identity.

Deliverable: CLI connected to AWS account.

Step 2: Cost Budget

- Created an AWS **budget alerts**
- Added email notifications to avoid unexpected charges.

Deliverable: Budget alert successfully configured.

Step 3: Cognito User Pool (Authentication)

- Created a **Cognito User Pool** named InventoryUsers.
- Enabled **Email sign-in** for user registration and login.
- Created an **App Client (no secret)** and activated the **Hosted UI** for login.
- Recorded:
    - **User Pool ID**
    - **App Client ID**

Deliverable: Working authentication system with user signup, email verification, and token generation.

Step 4: DynamoDB Tables

Created three tables using AWS CLI:

Table Name	Primary Key	Sort Key	Purpose
Items	ItemId	—	Store item details
StockBalances	LocationId	ItemId	Track stock per warehouse/location
Transactions	TransactionId	—	Record stock movement history

Enabled **PAY_PER_REQUEST** billing and **Point-in-Time Recovery (PITR)**.

Deliverable: DynamoDB tables operational.

Step 5: S3 Bucket

- Created private bucket yourname-inventory-<unique-id>.
- Enabled **default encryption** (SSE-S3).

- Blocked public access.
- Added folders: images/ and exports/.

Deliverable: Encrypted and secured S3 storage ready.

Step 6: IAM Role for Lambda

- Created **Lambda execution role** with:
  - AWSLambdaBasicExecutionRole policy
  - Inline policy allowing access to DynamoDB and S3 resources
- Attached this role to all Lambda functions.

Deliverable: Secure execution role created for Lambdas.

Step 7: Lambda Functions

Developed **five Lambda functions** in Node.js 22.x:

Function	Purpose
createItem	Insert new items into DynamoDB
getItems	Retrieve all items
adjustStock	Adjust stock quantities and record transactions atomically
getStock	Retrieve stock details for a specific location and item
presignUpload	Generate presigned URLs for S3 uploads

Configured environment variables:

ITEMS_TABLE=Items
STOCK_TABLE=StockBalances
TXN_TABLE=Transactions
IMAGES_BUCKET=yourname-inventory-bucket

Deliverable: All functions created, configured, and tested in Lambda console.

Step 8: API Gateway (HTTP API)

- Created **HTTP API** (not REST API).
- Added routes and connected each to its Lambda function:
  - POST /items
  - GET /items
  - POST /stock/adjust
  - GET /stock/{locationId}/{itemId}
  - GET /images/presign
- Configured **Cognito JWT Authorizer** with:
  - Issuer: https://cognito-idp.eu-west-3.amazonaws.com/<userPoolId>
  - Audience: <AppClientId>

- Enabled **CORS** for API testing.
- Deployed API and obtained HTTPS endpoint URL.

Deliverable: Secure HTTPS API deployed using API Gateway (HTTP API).

Step 9: Testing

- Signed in through **Cognito Hosted UI** to obtain id_token.
- Tested APIs via **curl** and **Postman**.

Example:

```
curl -X POST "https://<api-id>.execute-api.eu-west-3.amazonaws.com/stock/adjust" \
  -H "Authorization: Bearer <ID_TOKEN>" \
  -H "Content-Type: application/json" \
  -d '{"itemId":"item-001","locationId":"warehouse-1","qtyDelta":10}'
```

Verified:

- Successful DynamoDB updates
- Correct S3 presigned URL generation
- Proper JWT validation through API Gateway

Deliverable: End-to-end workflow validated.

Step 10: Monitoring, Logging, and Backups

- Verified **CloudWatch logs** for all Lambda functions.
- Configured **CloudWatch alarms** for:
    - Lambda error count > 1 per minute
    - API Gateway 5xx responses
- Enabled **Point-in-Time Recovery** for all DynamoDB tables.
- Verified **budget alert notifications**.

Deliverable: Monitoring and recovery systems active.

6. Security Features

- IAM roles follow **least privilege principle**.
- **MFA** enabled for all admin accounts.
- **Cognito** protects all APIs with JWT authorization.
- **API Gateway (HTTP API)** serves traffic only via **HTTPS**.
- **Encryption at rest** enabled for S3 and DynamoDB.
- **TLS (HTTPS)** ensures encryption in transit.
- **CloudWatch + IAM logs** record all activity for auditing.

7. Results

- Implemented a fully functional **serverless backend** using AWS services.
- All API calls secured via **HTTPS + Cognito authentication**.
- Achieved reliable, scalable, and low-cost infrastructure with **zero servers**.
- Verified CRUD operations, stock management, and image uploads through the API.

8. Challenges & Learnings

Challenge	Solution / Learning
Understanding Cognito configuration	Used Hosted UI for easier sign-in setup.
API Gateway CORS errors	Enabled CORS in HTTP API routes.
DynamoDB key schema confusion	Used composite keys for StockBalances table.
IAM permission errors	Created a custom Lambda execution policy with least privilege.

10. Conclusion

This project successfully demonstrates the deployment of a **cloud-native inventory management backend** using **AWS serverless technologies**.
The use of **API Gateway (HTTP API)** over traditional REST APIs provided faster performance, lower cost, and simpler integration with **Lambda** and **Cognito**.
The final system is secure, scalable, and easy to extend into a full-featured application.